

# Week 14: Bash Scripting Basics (Part 2)

Date: June 8, 2026

Study Time: 5-8 hours total

---

## GOAL

Automate routine tasks with scripts and cron

---

## STUDY TASKS (1-2 hours)

### Reading

- ☐ "How Linux Works" - Chapter 11 (Advanced scripting)

### Video Learning

- ☐ LearnLinuxTV - "Bash Functions Arrays"
- ☐ Professor Messer - "Cron Jobs"

### Concepts

- ☐ Functions in bash
  - ☐ Arrays
  - ☐ Error handling (set -e, trap)
  - ☐ Debugging (set -x)
  - ☐ cron syntax
  - ☐ crontab management
  - ☐ Script logging
- 

## LAB WORK (3-4 hours)

### Functions

- ☐ Basic function

bash

```
nano ~/functions.sh
```

```
#!/bin/bash
```

```
# Define function
```

```
greet() {  
    echo "Hello, $1!"  
}
```

```
# Call function
```

```
greet "Dillon"
```

```
greet "World"
```

```
# Function with return value
```

```
add_numbers() {  
    local result=$(( $1 + $2 ))  
    echo $result  
}
```

```
sum=$(add_numbers 5 10)
```

```
echo "5 + 10 = $sum"
```

☐ **Practical function example**

bash

```
nano ~/system-functions.sh

#!/bin/bash

# Function to check if service is running
check_service() {
    local service=$1
    if systemctl is-active --quiet $service; then
        echo "✓ $service is running"
        return 0
    else
        echo "✗ $service is NOT running"
        return 1
    fi
}

# Function to update system
update_system() {
    echo "Updating system packages..."
    if [ -f /etc/debian_version ]; then
        sudo apt update && sudo apt upgrade -y
    elif [ -f /etc/fedora-release ]; then
        sudo dnf update -y
    fi
}

# Use functions
check_service sshd
check_service nginx

# Uncomment to actually update:
# update_system
```

## Arrays

### ☐ Declaring and using arrays

bash

```
nano ~/arrays.sh

#!/bin/bash

# Declare array
devices=("laptop" "pi4" "pi-zero" "fedora")

# Access elements
echo "First device: ${devices[0]}"
echo "Second device: ${devices[1]}"

# All elements
echo "All devices: ${devices[@]}"

# Number of elements
echo "Total devices: ${#devices[@]}"

# Loop through array
for device in "${devices[@]"; do
    echo "Processing: $device"
done

# Add element
devices+=("new-device")
echo "Updated array: ${devices[@]}"
```

☐ **Practical: Ping all devices**

bash

```
nano ~/ping-all.sh

#!/bin/bash

# Array of Tailscale IPs
declare -a ips=(
    "100.64.0.10" # Fedora
    "100.64.0.20" # Pi Zero
    "100.64.0.30" # Pi 4
)

# Corresponding names
declare -a names=(
    "Fedora Server"
    "Pi Zero (Bastion)"
    "Pi 4 (Automation)"
)

echo "=== Device Health Check ==="
echo "Started: $(date)"
echo ""

for i in "${!ips[@]"; do
    ip="${ips[$i]}"
    name="${names[$i]}"

    if ping -c 1 -W 2 $ip > /dev/null 2>&1; then
        echo "✓ $name ($ip) is UP"
    else
        echo "✗ $name ($ip) is DOWN"
    fi
done

chmod +x ~/ping-all.sh
```

## Error Handling

☐ Using set -e

bash

```
nano ~/error-handling.sh

#!/bin/bash
set -e # Exit immediately if any command fails

echo "Starting..."

ls /home # This works
echo "First command succeeded"

ls /nonexistent # This fails, script exits here
echo "This line never executes"
```

## ☐ Using trap

bash

```
nano ~/trap-example.sh

#!/bin/bash

# Cleanup function
cleanup() {
    echo ""
    echo "Script interrupted or completed"
    echo "Cleaning up..."
    # Add cleanup commands here
}

# Run cleanup on exit or interrupt
trap cleanup EXIT
trap 'echo "Error on line $LINENO"' ERR

echo "Script running..."
sleep 2
echo "Still running... (try Ctrl+C)"
sleep 2
echo "Done"
```

## ☐ Comprehensive error handling

bash

```
nano ~/robust-script.sh

#!/bin/bash
set -euo pipefail # Exit on error, undefined vars, pipe failures

# Error handler
error_exit() {
    echo "Error: $1" >&2
    exit 1
}

# Check if running as root
if [ "$EUID" -ne 0 ]; then
    error_exit "This script must be run as root"
fi

# Check if file exists
CONFIG_FILE="/etc/myconfig"
if [ ! -f "$CONFIG_FILE" ]; then
    error_exit "Config file not found: $CONFIG_FILE"
fi

echo "All checks passed"
```

## Debugging Scripts

☐ Using set -x

bash

```
nano ~/debug-example.sh

#!/bin/bash
set -x  # Print each command before executing

NAME="Dillon"
AGE=31

echo "Name: $NAME"
echo "Age: $AGE"

if [ $AGE -gt 30 ]; then
    echo "Over 30"
fi

# Turn off debugging
set +x

echo "This line is not debugged"
```

## Automation Script: Update All Devices

☐ Create master update script



bash

```
nano ~/update-all-devices.sh

#!/bin/bash
set -euo pipefail

# Array of devices
declare -a devices=(
    "fedora"
    "pi4"
    "pi-zero"
)

# Function to update a device
update_device() {
    local device=$1
    echo "====="
    echo "Updating: $device"
    echo "====="

    ssh $device 'bash -s' << 'ENDSSH'
# Commands to run on remote device

    echo "Starting update on $(hostname)..."

# Detect OS and update
    if [ -f /etc/debian_version ]; then
        sudo apt update
        sudo apt upgrade -y
        sudo apt autoremove -y
    elif [ -f /etc/fedora-release ]; then
        sudo dnf update -y
        sudo dnf autoremove -y
    fi

    echo "Update completed on $(hostname)"
    ENDSSH

    if [ $? -eq 0 ]; then
        echo "✓ $device updated successfully"
    else
        echo "✗ $device update failed"
    fi
    echo ""
}
```

```
# Main execution
echo "=== Automated System Update ==="
echo "Started: $(date)"
echo ""

for device in "${devices[@]}; do
    update_device $device
done

echo "=== All Updates Complete ==="
echo "Finished: $(date)"

chmod +x ~/update-all-devices.sh
```

## Introduction to Cron

### ☐ Understanding cron syntax

```

|_____ minute (0 - 59)
| |_____ hour (0 - 23)
| | |_____ day of month (1 - 31)
| | | |_____ month (1 - 12)
| | | | |_____ day of week (0 - 6) (Sunday=0)
| | | | |
* * * * * command to execute
```

Examples:

```
0 2 * * *    # Every day at 2:00 AM
*/15 * * * * # Every 15 minutes
0 0 * * 0    # Every Sunday at midnight
0 9 * * 1-5  # Weekdays at 9:00 AM
```

### ☐ View current crontab

bash

```
crontab -l
# Shows your current cron jobs
# Might be empty if you haven't set any
```

### ☐ Edit crontab

bash

```
crontab -e  
# Opens editor (usually nano or vim)
```

## ☐ Add simple cron job

bash

```
crontab -e  
  
# Add this line (runs every 5 minutes):  
*/5 * * * * /bin/date >> /tmp/cron-test.log  
  
# Save and exit  
  
# Wait 5 minutes, then check:  
cat /tmp/cron-test.log
```

## Practical Cron Jobs

### ☐ Daily backup at 2 AM

bash

```
crontab -e  
  
# Add:  
0 2 * * * /home/username/backup-gitea.sh >> /var/log/backup.log 2>&1
```

### ☐ Health check every hour

bash

```
crontab -e  
  
# Add:  
0 * * * * /home/username/ping-all.sh >> /var/log/health-check.log 2>&1
```

### ☐ Update all devices weekly (Sunday 3 AM)

bash

```
crontab -e
```

```
# Add:
```

```
0 3 * * 0 /home/username/update-all-devices.sh >> /var/log/auto-update.log 2>&1
```

## Check Cron Logs

### ☐ View cron execution logs

bash

```
# Ubuntu/Debian:
```

```
grep CRON /var/log/syslog
```

```
# Fedora:
```

```
grep CRON /var/log/cron
```

```
# Or with journalctl:
```

```
journalctl -u cron
```

```
journalctl -u crond # Fedora
```

## Create Log Analysis Script

### ☐ Script to analyze your logs

bash

```
nano ~/analyze-logs.sh
```

```
#!/bin/bash
```

```
LOGFILE="/var/log/messages"
```

```
OUTPUT="/tmp/log-report-$(date +%Y%m%d).txt"
```

```
echo "=== Log Analysis Report ===" > $OUTPUT
```

```
echo "Generated: $(date)" >> $OUTPUT
```

```
echo "" >> $OUTPUT
```

```
echo "--- Error Count ---" >> $OUTPUT
```

```
grep -i "error" $LOGFILE | wc -l >> $OUTPUT
```

```
echo "" >> $OUTPUT
```

```
echo "--- Warning Count ---" >> $OUTPUT
```

```
grep -i "warning" $LOGFILE | wc -l >> $OUTPUT
```

```
echo "" >> $OUTPUT
```

```
echo "--- Failed Login Attempts ---" >> $OUTPUT
```

```
grep "Failed password" /var/log/secure 2>/dev/null | wc -l >> $OUTPUT
```

```
echo "" >> $OUTPUT
```

```
echo "--- Top 10 Most Common Messages ---" >> $OUTPUT
```

```
awk '{for(i=6;i<=NF;i++) printf "%s ", $i; print ""}' $LOGFILE | \
    sort | uniq -c | sort -nr | head -10 >> $OUTPUT
```

```
echo "Report saved to: $OUTPUT"
```

```
cat $OUTPUT
```

```
chmod +x ~/analyze-logs.sh
```

---

## FLASHCARD COMMANDS

1. `cron`
  2. `crontab -e`
  3. `crontab -l`
  4. `crontab -r`
  5. `at`
  6. `function`
  7. `set -e`
  8. `set -x`
  9. `trap`
  10. `array[@]`
  11. `${#array[@]}`
  12. `local`
- 

## EXAM OBJECTIVES

[https://partners.comptia.org/docs/default-source/resources/comptia-linux-xk0-005-exam-objectives-\(3-0\)](https://partners.comptia.org/docs/default-source/resources/comptia-linux-xk0-005-exam-objectives-(3-0))

---

## NOTES

### Scripts created:

- ☐ update-all-devices.sh
- ☐ ping-all.sh
- ☐ analyze-logs.sh

### Cron jobs scheduled:

- ☐ Daily backup
- ☐ Hourly health check
- ☐ Weekly updates

Completed: ☐

---

## QUICK REFERENCE

### Cron Syntax:

```
* * * * * command
| | | | |
| | | | └─ Day of week (0-6, 0=Sunday)
| | | └─── Month (1-12)
| | └───── Day of month (1-31)
| └──────── Hour (0-23)
└────────── Minute (0-59)
```

Special characters:

```
* = any value
, = list (1,3,5)
- = range (1-5)
/ = step (* / 5 = every 5)
```

## Common Cron Schedules:

```
bash

*/5 * * * *      # Every 5 minutes
0 * * * *        # Every hour
0 2 * * *        # 2 AM daily
0 0 * * 0        # Midnight Sunday
0 9 * * 1-5      # 9 AM weekdays
@reboot          # On system boot
@daily           # Once a day (0:00)
@weekly          # Once a week (Sunday 0:00)
@monthly         # Once a month (1st, 0:00)
```

---

**Previous Week:** Basic Scripting

**Next Week:** Package Management (apt)